

SIH 2026 — Internal Practical Assessment

SADHANA · Reg 411724106067 · PSVPEC · ECE · Year III

Level: Easy

Duration: 2 days

Marks: 70

Problem Statement — Godown Fire and Smoke Early Detection Node

Problem Description

A godown storing fertiliser, fodder or packaged goods is unattended at night. A fire that begins as smouldering material is detected only when it is already alight and visible from outside, by which time the stock and often the structure are lost. Detection equipment exists but is priced for industrial buildings.

Objective

To build a low-cost detection node that identifies smoke and a rapid temperature rise, raises an immediate local alarm without depending on the network, and records the readings leading up to the event.

Tools You May Use

Wokwi or Tinkercad with ESP32 or Arduino. SIMULATION ONLY.

Step-by-Step Tasks

Task 1: Define the Requirements and Generate the Test Values

Write a requirement table fixing the sampling interval, the safe range for the quantity you are measuring, the threshold at which action is needed, and how many consecutive readings you will require before acting. Then generate three signals in your simulator: normal values, a genuine excursion, and a noisy signal containing isolated spikes and one stuck value.

Logic: These values are the design decisions of the whole project and the three signals are the evidence you will test against, so fixing them first is what lets every later task be demonstrated rather than merely described.

Task 2: Build the Sensing Node

In your simulator, read smoke level and temperature on a fixed schedule using non-blocking timing rather than a blocking delay. Apply a plausibility check that rejects impossible values and smooth the reading so a single spike cannot be mistaken for a real change.

Logic: A blocking delay freezes everything else including the network, and an unfiltered spike produces false alarms that cause the whole system to be switched off. Both are the standard reasons field devices fail in practice.

Task 3: Implement the Local Decision and Store-and-Forward

Raise the alert locally when the reading stays outside the safe range for several consecutive readings, working without any network. Publish readings when the network is available, and when it is not, hold them with the time they were taken and send them oldest-first on reconnection so nothing is lost.

Logic: The alert must work when the network is down, because that is exactly when trouble tends to occur. Timestamping at the moment of reading rather than of sending is what keeps the recovered history truthful.

Task 4: Handle Sensor Failure and Prove It With Tests

Detect a failed sensor — a value stuck unchanged, outside the plausible range, or not updating — and enter a clear fault state rather than acting on the reading. Then run and record five cases: normal, a genuine excursion, a noisy signal, a stuck sensor, and a network outage followed by reconnection.

Logic: A stuck reading is not a measurement, and treating it as one is how a device causes harm while appearing to work. Each behaviour can break independently, so all five cases must be recorded together.

Task 5: Integrate and Test the Whole Thing

Connect every part and test it properly: confirm the main flow works from start to finish with your own data; run the normal, extreme and faulty cases and record the log for each, then disconnect the network and confirm the local behaviour still works; check one calculated figure by hand against your data. Handle the loading, empty and error states so no screen or output is ever blank or fails silently.

Logic: Each part can work perfectly alone while the whole fails at the joins, and this is the only test that proves otherwise. Checking a figure by hand is how you know the number the officer in charge will act on is actually right.

Task 6: Document and Demonstrate

Write a README stating the problem in two lines, how to run your work step by step, what every field means, how any derived figure is calculated, and what is not finished. Take three or four screenshots of your own working solution, record a short demonstration video, and upload everything to one public GitHub repository.

Logic: Somebody else must be able to run and understand this without you sitting beside them. Explaining how a derived figure is calculated matters most, because that is the number people will act on.

Please note: this is an Easy-level assessment. A simple, clear solution that works is exactly what is expected — advanced features are not required. Every task builds on the previous one, so complete them in order. Submit everything through one public GitHub repository. If you cannot complete a task, submit what you have and state in one line what remains.